



SPIR-V Extended Instructions for GLSL

John Kessenich, Google

Version 1.00, Revision 17

Table of Contents

1. Introduction	3
2. Binary Form	4
3. Appendix A: Changes	30
3.1. Changes from Version 0.99, Revision 1	30
3.2. Changes from Version 0.99, Revision 2	30
3.3. Changes from Version 0.99, Revision 3	30
3.4. Changes from Version 1.00, Revision 1	30
3.5. Changes from Version 1.00, Revision 2	30
3.6. Changes from Version 1.00, Revision 3	30
3.7. Changes from Version 1.00, Revision 4	31
3.8. Changes from Version 1.00, Revision 5	31
3.9. Changes from Version 1.00, Revision 6	31
3.10. Changes from Version 1.00, Revision 7	31
3.11. Changes from Version 1.00, Revision 8	31
3.12. Changes from Version 1.00, Revision 9	31
3.13. Changes from Version 1.00, Revision 10	31
3.14. Changes from Version 1.00, Revision 11	31
3.15. Changes from Version 1.00, Revision 12	32
3.16. Changes from Version 1.00, Revision 13	32
3.17. Changes from Version 1.00, Revision 14	32
3.18. Changes from Version 1.00, Revision 15	32
3.19. Changes from Version 1.00, Revision 16	32



Copyright 2014-2026 The Khronos Group Inc.

This Specification is protected by copyright laws and contains material proprietary to Khronos. Except as described by these terms, it or any components may not be reproduced, republished, distributed, transmitted, displayed, broadcast or otherwise exploited in any manner without the express prior written permission of Khronos.

Khronos grants a conditional copyright license to use and reproduce the unmodified Specification for any purpose, without fee or royalty, EXCEPT no licenses to any patent, trademark or other intellectual property rights are granted under these terms.

Khronos makes no, and expressly disclaims any, representations or warranties, express or implied, regarding this Specification, including, without limitation: merchantability, fitness for a particular purpose, non-infringement of any intellectual property, correctness, accuracy, completeness, timeliness, and reliability. Under no circumstances will Khronos, or any of its Promoters, Contributors or Members, or their respective partners, officers, directors, employees, agents or representatives be liable for any damages, whether direct, indirect, special or consequential damages for lost revenues, lost profits, or otherwise, arising from or in connection with these materials.

This Specification has been created under the Khronos Intellectual Property Rights Policy, which is Attachment A of the Khronos Group Membership Agreement available at https://www.khronos.org/files/member_agreement.pdf. Parties desiring to implement the Specification and make use of Khronos trademarks in relation to that implementation, and receive reciprocal patent license protection under the Khronos Intellectual Property Rights Policy must become Adopters and confirm the implementation as conformant under the process defined by Khronos for this Specification; see <https://www.khronos.org/adopters>.

This Specification contains substantially unmodified functionality from, and is a successor to, Khronos specifications including all versions of "The SPIR Specification", "The OpenGL Shading Language", and "The OpenGL ES Shading Language", as well as all Khronos OpenCL API and OpenCL programming language specifications.

The Khronos Intellectual Property Rights Policy defines the terms 'Scope', 'Compliant Portion', and 'Necessary Patent Claims'.

Where this Specification uses technical terminology, defined in the Glossary or otherwise, that refer to enabling technologies that are not expressly set forth in this Specification, those enabling technologies are EXCLUDED from the Scope of this Specification. For clarity, enabling technologies not disclosed with particularity in this Specification (e.g. semiconductor manufacturing technology, hardware architecture, processor architecture or microarchitecture, memory architecture, compiler technology, object oriented technology, basic operating system technology, compression technology, algorithms, and so on) are NOT to be considered expressly set forth; only those application program interfaces and data structures disclosed with particularity are included in the Scope of this Specification.

For purposes of the Khronos Intellectual Property Rights Policy as it relates to the definition of Necessary Patent Claims, all recommended or optional features, behaviors and functionality set forth in this Specification, if implemented, are considered to be included as Compliant Portions.

Where this Specification identifies specific sections of external references, only those specifically identified sections define normative functionality. The Khronos Intellectual Property Rights Policy excludes external references to materials and associated enabling technology not created by Khronos from the Scope of this Specification, and any licenses that may be required to implement such referenced materials and

associated technologies must be obtained separately and may involve royalty payments.

Khronos and Vulkan are registered trademarks, and SPIR, SPIR-V, and SYCL are trademarks of The Khronos Group Inc. OpenCL is a trademark of Apple Inc., used under license by Khronos. OpenGL is a registered trademark and the OpenGL ES logo is a trademark of Hewlett Packard Enterprise, used under license by Khronos. All other product names, trademarks, and/or company names are used solely for identification and belong to their respective owners.

Chapter 1. Introduction

This specifies the GLSL.std.450 extended instruction set. It provides instructions for the GLSL built-in functions that do not directly map to native SPIR-V instructions.

Import this extended instruction set using an **OpExtInstImport** "GLSL.std.450" instruction.

Where requirements are written in the form $f(\pm x) = \pm y$, this means that $f(x) = y$ and $f(-x) = -y$.

In this specification, a floating-point type is any **OpTypeFloat** using the IEEE 754 encoding.

NOTE

These instructions may specify their behavior for special floating-point values such as -0.0, Infinities or NaN. Such values can only be reliably observed when **SPV_KHR_float_controls2** is used because the **SignedZeroInfNaNPreserve** execution mode does not apply to these instructions and by default these special values may be ignored. In some cases, client APIs may allow an error bound for results which may include values other than the reference value given.

Chapter 2. Binary Form

Documentation form for each extended instruction:

Extended Instruction Name

Instruction description.

Result Type will describe the Result Type for the **OpExtInst** instruction.

Number is the extended instruction number to use in the **OpExtInst** instruction.

Operand 1, Operand 2,... are the operands listed for the **OpExtInst** instruction.

Any **Capability** restrictions.

Number	Operand 1	Operand 2	...
--------	-----------	-----------	-----

Extended instructions:

Round

Result is the value equal to the nearest whole number to *x*. The fraction 0.5 rounds in a direction chosen by the implementation, presumably the direction that is fastest. This includes the possibility that **Round** *x* is the same value as **RoundEven** *x* for all values of *x*.

round(± 0) = ± 0

round($\pm \text{Inf}$) = $\pm \text{Inf}$.

The operand *x* must be a scalar or vector whose component type is floating-point.

Result Type and the type of *x* must be the same type. Results are computed per component.

1	<id> <i>x</i>
---	------------------

RoundEven

Result is the value equal to the nearest whole number to *x*. A fractional part of 0.5 rounds toward the nearest even whole number. (Both 3.5 and 4.5 for *x* round to 4.0.)

roundEven(± 0) = ± 0 .

roundEven($\pm \text{Inf}$) = $\pm \text{Inf}$.

The operand *x* must be a scalar or vector whose component type is floating-point.

Result Type and the type of *x* must be the same type. Results are computed per component.

2	<id> x
---	------------------------------------

Trunc

Result is the value equal to the nearest whole number to *x* whose absolute value is not larger than the absolute value of *x*.

trunc(±0) = ±0.

trunc(±Inf) = ±Inf.

The operand *x* must be a scalar or vector whose component type is floating-point.

Result Type and the type of *x* must be the same type. Results are computed per component.

3	<id> x
---	------------------------------------

FAbs

Result is +0.0 if *x* is ±0.0, *x* if *x* > 0.0, and -*x* if *x* < 0.0.

fabs(±0) = +0.0.

fabs(±Inf) = ±Inf.

The operand *x* must be a scalar or vector whose component type is floating-point.

Result Type and the type of *x* must be the same type. Results are computed per component.

4	<id> x
---	------------------------------------

SAbs

Result is *x* if *x* >= 0; otherwise result is -*x*, where *x* is interpreted as a signed integer.

Result Type and the type of *x* must both be integer scalar or integer vector types. *Result Type* and operand types must have the same number of components with the same component width. Results are computed per component.

This instruction can be decorated with **NoSignedWrap**.

5	<id> x
---	------------------------------------

FSign

Result is 1.0 if $x > 0$, -1.0 if $x < 0$, $+0.0$ if $x = +0.0$, and ± 0.0 if $x = -0.0$. $\text{fsign}(\pm\text{Inf}) = \pm 1$ If $x = \pm\text{NaN}$, the result can be any of ± 1.0 or ± 0.0 , regardless of whether `shader_float_controls` is in use.

The operand x must be a scalar or vector whose component type is floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

6	<code><id></code> x
---	--------------------------------

SSign

Result is 1 if $x > 0$, 0 if $x = 0$, or -1 if $x < 0$, where x is interpreted as a signed integer.

Result Type and the type of x must both be integer scalar or integer vector types. *Result Type* and operand types must have the same number of components with the same component width. Results are computed per component.

7	<code><id></code> x
---	--------------------------------

Floor

Result is the value equal to the nearest whole number that is less than or equal to x .

$\text{floor}(\pm 0) = \pm 0$.

$\text{floor}(\pm\text{Inf}) = \pm\text{Inf}$.

The operand x must be a scalar or vector whose component type is floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

8	<code><id></code> x
---	--------------------------------

Ceil

Result is the value equal to the nearest whole number that is greater than or equal to x .

$\text{ceil}(\pm 0) = \pm 0$.

$\text{ceil}(\pm\text{Inf}) = \pm\text{Inf}$.

The operand x must be a scalar or vector whose component type is floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

9	<code><id></code> x
---	--------------------------------

Fract

Result is $x - \text{floor } x$.

$\text{fract}(\pm 0) = +0$.

$\text{fract}(\pm \text{Inf}) = \text{NaN}$.

The operand x must be a scalar or vector whose component type is floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

10

<id>
 x

Radians

Converts *degrees* to radians, i.e., $\text{degrees} * \pi / 180$.

The operand *degrees* must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of *degrees* must be the same type. Results are computed per component.

11

<id>
degrees

Degrees

Converts *radians* to degrees, i.e., $\text{radians} * 180 / \pi$.

The operand *radians* must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of *radians* must be the same type. Results are computed per component.

12

<id>
radians

Sin

The standard trigonometric sine of x radians.

$\sin(\pm \text{Inf}) = \text{NaN}$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

13

<id>
 x

Cos

The standard trigonometric cosine of x radians.

$\cos(\pm\text{Inf}) = \text{NaN}$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

14

<id>
 x

Tan

The standard trigonometric tangent of x radians.

$\tan(\pm\text{Inf}) = \text{NaN}$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

15

<id>
 x

Asin

Arc sine. Result is an angle, in radians, whose sine is x . The range of result values is $[-\pi / 2, \pi / 2]$. The resulting value is NaN if **abs** $x > 1$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

16

<id>
 x

Acos

Arc cosine. Result is an angle, in radians, whose cosine is x . The range of result values is $[0, \pi]$. The resulting value is NaN if **abs** $x > 1$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

17

<id>
 x

Atan

Arc tangent. Result is an angle, in radians, whose tangent is y_over_x . The range of result values is $[-\pi / 2, \pi / 2]$.

The operand y_over_x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of y_over_x must be the same type. Results are computed per component.

18	$\langle id \rangle$ y_over_x
----	--------------------------------------

Sinh

Hyperbolic sine of x radians.

$\sinh(\pm Inf) = \pm Inf$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

19	$\langle id \rangle$ x
----	-----------------------------

Cosh

Hyperbolic cosine of x radians.

$\cosh(\pm Inf) = Inf$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

20	$\langle id \rangle$ x
----	-----------------------------

Tanh

Hyperbolic tangent of x radians.

$\tanh(\pm Inf) = \pm 1$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

21	$\langle id \rangle$ x
----	-----------------------------

Asinh

Arc hyperbolic sine; result is the inverse of **sinh**.

$\text{asinh}(\pm\text{Inf}) = \pm\text{Inf}$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

22	<i><id></i> x
----	--------------------------

Acosh

Arc hyperbolic cosine; Result is the non-negative inverse of **cosh**. The resulting value is NaN if $x < 1$.

$\text{acosh}(\text{Inf}) = \text{Inf}$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

23	<i><id></i> x
----	--------------------------

Atanh

Arc hyperbolic tangent; result is the inverse of tanh. The resulting value is NaN if **abs** $x \geq 1$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

24	<i><id></i> x
----	--------------------------

Atan2

Arc tangent. Result is an angle, in radians, whose tangent is y / x . The signs of x and y are used to determine what quadrant the angle is in. The range of result values is $[-\pi, \pi]$. The resulting value is *poison* if x and y are both 0.

The operand x and y must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of all operands must be the same type. Results are computed per component.

25	<i><id></i> y	<i><id></i> x
----	--------------------------	--------------------------

Pow

Result is x raised to the y power; x^y . The resulting value is *poison* if $x < 0$. Result is *poison* if $x = 0$ and $y \leq 0$.

The operand x and y must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of all operands must be the same type. Results are computed per component.

26

<id>
 x

<id>
 y

Exp

Result is the natural exponentiation of x ; e^x .

$\exp(\text{Inf}) = \text{Inf}$.

$\exp(-\text{Inf}) = +0$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

27

<id>
 x

Log

Result is the natural logarithm of x , i.e., the value y which satisfies the equation $x = e^y$. The resulting value is NaN if $x < 0$.

$\log(\text{Inf}) = \text{Inf}$.

$\log(1.0) = +0$.

$\log(\pm 0) = -\text{Inf}$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

28

<id>
 x

Exp2

Result is 2 raised to the x power; 2^x .

$\text{exp2}(\text{Inf}) = \text{Inf}$.

$\text{exp2}(-\text{Inf}) = +0$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

29

<id>

x

Log2

Result is the base-2 logarithm of x , i.e., the value y which satisfies the equation $x = 2^y$. The resulting value is NaN if $x < 0$.

$\log(\text{Inf}) = \text{Inf}$.

$\log(1.0) = +0$.

$\log(\pm 0) = -\text{Inf}$.

The operand x must be a scalar or vector whose component type is 16-bit or 32-bit floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

30

<id>

x

Sqrt

Result is the square root of x . The resulting value is NaN if $x < 0$.

$\text{sqrt}(\text{Inf}) = \text{Inf}$.

$\text{sqrt}(\pm 0) = \pm 0$.

The operand x must be a scalar or vector whose component type is floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

31

<id>

x

InverseSqrt

Result is the reciprocal of **sqrt** x . The resulting value is NaN if $x < 0$.

`inversesqrt(Inf) = +0`.

`inversesqrt( $\pm 0$ ) =  $\pm$ Inf`.

The operand x must be a scalar or vector whose component type is floating-point.

Result Type and the type of x must be the same type. Results are computed per component.

32

<id>
 x

Determinant

Result is the determinant of x .

The operand x must be a square matrix.

Result Type must be the same type as the component type in the columns of x .

33

<id>
 x

MatrixInverse

Result is a matrix that is the inverse of x . The resulting values are *poison* if x is singular or poorly conditioned (nearly singular).

The operand x must be a square matrix.

Result Type and the type of x must be the same type.

34

<id>
 x

Modf

Modf is deprecated, use **ModfStruct** instead.

Result is the fractional part of x , and stores through i the whole-number part as a whole-number floating-point value. Both the result and the output parameter have the same sign as x .

The operand x must be a scalar or vector whose component type is floating-point.

The operand i must have a pointer type.

Result Type, the type of x , and the type i points to must all be the same type and have a floating-point component type. Results are computed per component.

35	$\langle id \rangle$ x	$\langle id \rangle$ i
----	-----------------------------	-----------------------------

ModfStruct

Result is a structure containing both the fractional part of x and the whole number part of x .

Result Type must be an **OpTypeStruct** with two members. Member 0 holds the fractional part. Member 1 holds the whole number part. Both members get the same sign as x .

$\text{modf}(\pm 0) = \{ \pm 0, \pm 0 \}$.

$\text{modf}(\pm \text{Inf}) = \{ \pm 0, \pm \text{Inf} \}$.

The two members of the returned struct and x must all be the same type. Results are computed per component.

The operand x must be a scalar or vector whose component type is floating-point.

36	$\langle id \rangle$ x	
----	-----------------------------	--

FMin

Result is y if $y < x$, otherwise x . -0 compares less than $+0$. Which operand is the result if one of the operands is a NaN is implementation-dependent.

The operands must all be a scalar or vector whose component type is floating-point.

Result Type and the type of all operands must be the same type. Results are computed per component.

37	$\langle id \rangle$ x	$\langle id \rangle$ y
----	-----------------------------	-----------------------------

UMin

Result is y if $y < x$; otherwise result is x , where x and y are interpreted as unsigned integers.

Result Type and the type of x and y must both be integer scalar or integer vector types. *Result Type* and operand types must have the same number of components with the same component width. Results are computed per component.

38	$\langle id \rangle$ x	$\langle id \rangle$ y
----	-----------------------------	-----------------------------

SMin

Result is y if $y < x$; otherwise result is x , where x and y are interpreted as signed integers.

Result Type and the type of x and y must both be integer scalar or integer vector types. *Result Type* and operand types must have the same number of components with the same component width. Results are computed per component.

39	<code><id></code> <code>x</code>	<code><id></code> <code>y</code>
----	---	---

FMax

Result is `y` if `x < y`, otherwise `x`. `-0` compares less than `+0`. Which operand is the result if one of the operands is a NaN is implementation-dependent.

The operands must all be a scalar or vector whose component type is floating-point.

Result Type and the type of all operands must be the same type. Results are computed per component.

40	<code><id></code> <code>x</code>	<code><id></code> <code>y</code>
----	---	---

UMax

Result is `y` if `x < y`; otherwise result is `x`, where `x` and `y` are interpreted as unsigned integers.

Result Type and the type of `x` and `y` must both be integer scalar or integer vector types. *Result Type* and operand types must have the same number of components with the same component width. Results are computed per component.

41	<code><id></code> <code>x</code>	<code><id></code> <code>y</code>
----	---	---

SMax

Result is `y` if `x < y`; otherwise result is `x`, where `x` and `y` are interpreted as signed integers.

Result Type and the type of `x` and `y` must both be integer scalar or integer vector types. *Result Type* and operand types must have the same number of components with the same component width. Results are computed per component.

42	<code><id></code> <code>x</code>	<code><id></code> <code>y</code>
----	---	---

FClamp

Result is `min(max(x, minVal), maxVal)`. The resulting value is *poison* if `minVal > maxVal`. The semantics used by `min()` and `max()` are those of `FMin` and `FMax`.

The operands must all be a scalar or vector whose component type is floating-point.

Result Type and the type of all operands must be the same type. Results are computed per component.

43	<code><id></code> <code>x</code>	<code><id></code> <code>minVal</code>	<code><id></code> <code>maxVal</code>
----	---	--	--

UClamp

Result is $\min(\max(x, \text{minVal}), \text{maxVal})$, where x , minVal and maxVal are interpreted as unsigned integers. The resulting value is *poison* if $\text{minVal} > \text{maxVal}$.

Result Type and the type of the operands must both be integer scalar or integer vector types. *Result Type* and operand types must have the same number of components with the same component width. Results are computed per component.

44	<id> x	<id> minVal	<id> maxVal
----	-------------	-------------------------	-------------------------

SClamp

Result is $\min(\max(x, \text{minVal}), \text{maxVal})$, where x , minVal and maxVal are interpreted as signed integers. The resulting value is *poison* if $\text{minVal} > \text{maxVal}$.

Result Type and the type of the operands must both be integer scalar or integer vector types. *Result Type* and operand types must have the same number of components with the same component width. Results are computed per component.

45	<id> x	<id> minVal	<id> maxVal
----	-------------	-------------------------	-------------------------

FMix

Result is the linear blend of x and y , i.e., $x * (1 - a) + y * a$.

The operands must all be a scalar or vector whose component type is floating-point.

Result Type and the type of all operands must be the same type. Results are computed per component.

46	<id> x	<id> y	<id> a
----	-------------	-------------	-------------

Step

Result is 0.0 if $x < \text{edge}$; otherwise result is 1.0.

The operands must all be a scalar or vector whose component type is floating-point.

Result Type and the type of all operands must be the same type. Results are computed per component.

48	<id> edge	<id> x
----	-----------------------	-------------

SmoothStep

Result is 0.0 if $x \leq \text{edge0}$ and 1.0 if $x \geq \text{edge1}$ and performs smooth Hermite interpolation between 0 and 1 if $\text{edge0} < x < \text{edge1}$. This is equivalent to:

$t * t * (3 - 2 * t)$, where $t = \text{clamp}((x - \text{edge0}) / (\text{edge1} - \text{edge0}), 0, 1)$

The resulting value is *poison* if $\text{edge0} \geq \text{edge1}$.

The operands must all be a scalar or vector whose component type is floating-point.

Result Type and the type of all operands must be the same type. Results are computed per component.

49	<id> edge0	<id> edge1	<id> x
----	---------------	---------------	-----------

Fma

Computes $a * b + c$.

The operands must all be a scalar or vector whose component type is floating-point.

Result Type and the type of all operands must be the same type. Results are computed per component.

50	<id> a	<id> b	<id> c
----	-----------	-----------	-----------

Frexp

Frexp is deprecated, use **FrexpStruct** instead.

Splits x into a floating-point significand in the range $(-1.0, 0.5]$ or $[0.5, 1.0)$ and an integral exponent of 2, such that:

$x = \text{significand} * 2^{\text{exponent}}$

The *significand* is the instruction result. An x of -0.0 results in a significand -0.0 , while an x of 0.0 results in 0.0 . For a floating-point value that is an infinity or is not a number, the significand is *poison*.

The operand x must be a scalar or vector whose component type is floating-point.

The exponent is returned through the pointer-parameter *exp*. The *exp* operand must be a pointer to a scalar or vector with integer component type, with 32-bit component width. The number of components in x and what *exp* points to must be the same. If x is a zero, the exponent is 0.0. If x is an infinity or a NaN, the exponent is *poison*.

Result Type must be the same type as the type of x . Results are computed per component.

51	<id> x	<id> exp
----	-----------	-------------

FrexpStruct

Result is a structure containing x split into a floating-point significand in the range $(-1.0, 0.5]$ or $[0.5, 1.0)$ and an integral exponent of 2, such that:

$$x = \text{significand} * 2^{\text{exponent}}$$

If x is a zero, the exponent is 0.0. If x is an infinity or a NaN, the exponent is *poison*. If x is 0.0, the significand is 0.0. If x is -0.0, the significand is -0.0

Result Type must be an **OpTypeStruct** with two members. Member 0 must have the same type as the type of x . Member 0 holds the significand. Member 1 must be a scalar or vector with integer component type, with 32-bit component width. Member 1 holds the exponent. These two members and x must have the same number of components.

The operand x must be a scalar or vector whose component type is floating-point.

52

<id>

x

Ldexp

Builds a floating-point number from x and the corresponding integral exponent of two in exp :

$$x * 2^{exp}$$

If this product is too large to be finitely represented in the floating-point type, the resulting value is *poison*. exp is interpreted as a signed integer. If exp is greater than +128 (single precision), +1024 (double precision) or +16 (half precision), the resulting value is *poison*. If exp is less than -126 (single precision), -1022 (double precision) or -14 (half precision), the result may be flushed to zero. Additionally, splitting the value into a significand and exponent using **frexp** and then reconstructing a floating-point value using **Ldexp** should yield the original input for zero and all finite non-denormalized values.

The operand x must be a scalar or vector whose component type is floating-point.

The exp operand must be a scalar or vector with integer component type. The number of components in x and exp must be the same.

Result Type must be the same type as the type of x . Results are computed per component.

53

<id>

x

<id>

exp

PackSnorm4x8

First, converts each component of the normalized floating-point value v into 8-bit integer values. These are then packed into the result.

The conversion for component c of v to fixed point is done as follows:

$\text{round}(\text{clamp}(c, -1, +1) * 127.0)$

The **RelaxedPrecision** Decoration only affects the conversion step of the instruction.

The first component of the vector is written to the least significant bits of the output; the last component is written to the most significant bits.

The v operand must be a vector of 4 components whose type is a 32-bit floating-point.

Result Type must be a 32-bit integer type.

54

$\langle id \rangle$
 v

PackUnorm4x8

First, converts each component of the normalized floating-point value v into 8-bit integer values. These are then packed into the result.

The conversion for component c of v to fixed point is done as follows:

$\text{round}(\text{clamp}(c, 0, +1) * 255.0)$

The **RelaxedPrecision** Decoration only affects the conversion step of the instruction.

The first component of the vector is written to the least significant bits of the output; the last component is written to the most significant bits.

The v operand must be a vector of 4 components whose type is a 32-bit floating-point.

Result Type must be a 32-bit integer type.

55

$\langle id \rangle$
 v

PackSnorm2x16

First, converts each component of the normalized floating-point value v into 16-bit integer values. These are then packed into the result.

The conversion for component c of v to fixed point is done as follows:

$\text{round}(\text{clamp}(c, -1, +1) * 32767.0)$

The **RelaxedPrecision** Decoration only affects the conversion step of the instruction.

The first component of the vector is written to the least significant bits of the output; the last component is written to the most significant bits.

The v operand must be a vector of 2 components whose type is a 32-bit floating-point.

Result Type must be a 32-bit integer type.

56

$\langle id \rangle$
 v

PackUnorm2x16

First, converts each component of the normalized floating-point value v into 16-bit integer values. These are then packed into the result.

The conversion for component c of v to fixed point is done as follows:

$\text{round}(\text{clamp}(c, 0, +1) * 65535.0)$

The **RelaxedPrecision** Decoration only affects the conversion step of the instruction.

The first component of the vector is written to the least significant bits of the output; the last component is written to the most significant bits.

The v operand must be a vector of 2 components whose type is a 32-bit floating-point.

Result Type must be a 32-bit integer type.

57

$\langle id \rangle$
 v

PackHalf2x16

Result is the unsigned integer obtained by converting the components of a two-component floating-point vector to the 16-bit **OpTypeFloat**, and then packing these two 16-bit integers into a 32-bit unsigned integer. The first vector component specifies the 16 least-significant bits of the result; the second component specifies the 16 most-significant bits.

The **RelaxedPrecision** Decoration only affects the conversion step of the instruction.

The *v* operand must be a vector of 2 components whose type is a 32-bit floating-point.

Result Type must be a 32-bit integer type.

58	<id> <i>v</i>
----	------------------

PackDouble2x32

Result is the double-precision value obtained by packing the components of *v* into a 64-bit value. If an IEEE 754 Inf or NaN is created, it will not signal, and the resulting floating-point value is unspecified. Otherwise, the bit-level representation of *v* is preserved. The first vector component specifies the 32 least significant bits; the second component specifies the 32 most significant bits.

The *v* operand must be a vector of 2 components whose type is a 32-bit integer.

Result Type must be a 64-bit floating-point scalar.

Use of this instruction requires declaration of the **Float64** capability.

59	<id> <i>v</i>
----	------------------

UnpackSnorm2x16

First, unpacks a single 32-bit unsigned integer *p* into a pair of 16-bit signed integers. Then, each component is converted to a normalized floating-point value to generate the result. The conversion for unpacked fixed-point value *f* to floating point is done as follows:

$\text{clamp}(f / 32767.0, -1, +1)$

The first component of the result is extracted from the least significant bits of the input; the last component is extracted from the most significant bits.

The **RelaxedPrecision** Decoration only affects the conversion step of the instruction.

The *p* operand must be a scalar with 32-bit integer type.

Result Type must be a vector of 2 components whose type is 32-bit floating point.

60	$\langle id \rangle$ p
----	-----------------------------

UnpackUnorm2x16

First, unpacks a single 32-bit unsigned integer p into a pair of 16-bit unsigned integers. Then, each component is converted to a normalized floating-point value to generate the result. The conversion for unpacked fixed-point value f to floating point is done as follows:

$$f / 65535.0$$

The first component of the result is extracted from the least significant bits of the input; the last component is extracted from the most significant bits.

The **RelaxedPrecision** Decoration only affects the conversion step of the instruction.

The p operand must be a scalar with 32-bit integer type.

Result Type must be a vector of 2 components whose type is 32-bit floating point.

61	$\langle id \rangle$ p
----	-----------------------------

UnpackHalf2x16

Result is the two-component floating-point vector with components obtained by unpacking a 32-bit unsigned integer into a pair of 16-bit values, interpreting those values as 16-bit floating-point numbers according to the OpenGL Specification, and converting them to 32-bit floating-point values. Subnormal numbers are either preserved or flushed to zero, consistently within an implementation.

The first component of the vector is obtained from the 16 least-significant bits of v ; the second component is obtained from the 16 most-significant bits of v .

The **RelaxedPrecision** Decoration only affects the conversion step of the instruction.

The v operand must be a scalar with 32-bit integer type.

Result Type must be a vector of 2 components whose type is 32-bit floating point.

62	$\langle id \rangle$ v
----	-----------------------------

UnpackSnorm4x8

First, unpacks a single 32-bit unsigned integer p into four 8-bit signed integers. Then, each component is converted to a normalized floating-point value to generate the result. The conversion for unpacked fixed-point value f to floating point is done as follows:

$\text{clamp}(f / 127.0, -1, +1)$

The first component of the result is extracted from the least significant bits of the input; the last component is extracted from the most significant bits.

The **RelaxedPrecision** Decoration only affects the conversion step of the instruction.

The p operand must be a scalar with 32-bit integer type.

Result Type must be a vector of 4 components whose type is 32-bit floating point.

63

$\langle id \rangle$
 p

UnpackUnorm4x8

First, unpacks a single 32-bit unsigned integer p into four 8-bit unsigned integers. Then, each component is converted to a normalized floating-point value to generate the result. The conversion for unpacked fixed-point value f to floating point is done as follows:

$f / 255.0$

The first component of the result is extracted from the least significant bits of the input; the last component is extracted from the most significant bits.

The **RelaxedPrecision** Decoration only affects the conversion step of the instruction.

The p operand must be a scalar with 32-bit integer type.

Result Type must be a vector of 4 components whose type is 32-bit floating point.

64

$\langle id \rangle$
 p

UnpackDouble2x32

Result is the two-component unsigned integer vector representation of v . The bit-level representation of v is preserved. The first component of the vector contains the 32 least significant bits of the double; the second component consists of the 32 most significant bits.

The v operand must be a scalar whose type is 64-bit floating point.

Result Type must be a vector of 2 components whose type is a 32-bit integer.

Use of this instruction requires declaration of the **Float64** capability.

65	$\langle id \rangle$ v
----	-----------------------------

Length

Result is the length of vector x , i.e., $\sqrt{x[0]^2 + x[1]^2 + \dots}$.

The operand x must be a scalar or vector whose component type is floating-point.

Result Type must be a scalar of the same type as the component type of x .

66	$\langle id \rangle$ x
----	-----------------------------

Distance

Result is the distance between $p0$ and $p1$, i.e., $\text{length}(p0 - p1)$.

The operands must all be a scalar or vector whose component type is floating-point.

Result Type must be a scalar of the same type as the component type of the operands.

67	$\langle id \rangle$ $p0$	$\langle id \rangle$ $p1$
----	------------------------------	------------------------------

Cross

Result is the cross product of x and y , i.e., the resulting components are, in order:

$$x[1] * y[2] - y[1] * x[2]$$

$$x[2] * y[0] - y[2] * x[0]$$

$$x[0] * y[1] - y[0] * x[1]$$

All the operands must be vectors of 3 components of a floating-point type.

Result Type and the type of all operands must be the same type.

68	$\langle id \rangle$ x	$\langle id \rangle$ y
----	-----------------------------	-----------------------------

Normalize

Result is the vector in the same direction as x but with a length of 1.

The operand x must be a scalar or vector whose component type is floating-point.

Result Type and the type of x must be the same type.

69	$\langle id \rangle$ x
----	-----------------------------

FaceForward

If the dot product of $Nref$ and I is negative, the result is N , otherwise it is $-N$.

The operands must all be a scalar or vector whose component type is floating-point.

Result Type and the type of all operands must be the same type.

70	$\langle id \rangle$ N	$\langle id \rangle$ I	$\langle id \rangle$ $Nref$
----	-----------------------------	-----------------------------	--------------------------------

Reflect

For the incident vector I and surface orientation N , returns $I - 2 * \text{dot}(N, I) * N$.

If N is normalized then this corresponds to I reflected from a surface with normal N .

The operands must all be a scalar or vector whose component type is floating-point.

Result Type and the type of all operands must be the same type.

71	$\langle id \rangle$ I	$\langle id \rangle$ N
----	-----------------------------	-----------------------------

Refract

For the incident vector I and surface normal N , and the ratio of indices of refraction η , the result is the refraction vector. The result is computed by

$$k = 1.0 - \eta * \eta * (1.0 - \text{dot}(N, I) * \text{dot}(N, I))$$

if $k < 0.0$ the result is 0.0

otherwise, the result is $\eta * I - (\eta * \text{dot}(N, I) + \text{sqrt}(k)) * N$

This computation assumes the input parameters for the incident vector I and the surface normal N are already normalized.

The type of I and N must be a scalar or vector with a floating-point component type.

The type of η must be a floating-point scalar.

Result Type, the type of I , the type of N , and the type of η must all have the same component type.

72	$\langle id \rangle$ I	$\langle id \rangle$ N	$\langle id \rangle$ η
----	-----------------------------	-----------------------------	--------------------------------

FindILsb

Integer least-significant bit.

Results in the bit number of the least-significant 1-bit in the binary representation of *Value*. If *Value* is 0, the result has all bits set (e.g., -1 if interpreted as signed).

Result Type and the type of *Value* must both be integer scalar or integer vector types. *Result Type* and operand types must have the same number of components with the same component width. Results are computed per component.

This instruction is currently limited to 32-bit width components.

73	$\langle id \rangle$ <i>Value</i>
----	--------------------------------------

FindSMsb

Signed-integer most-significant bit, with *Value* interpreted as a signed integer.

For positive numbers, the result is the bit number of the most significant 1-bit. For negative numbers, the result is the bit number of the most significant 0-bit. For a *Value* of 0 or -1, the result has all bits set (e.g., -1 if interpreted as signed).

Result Type and the type of *Value* must both be integer scalar or integer vector types. *Result Type* and operand types must have the same number of components with the same component width. Results are computed per component.

This instruction is currently limited to 32-bit width components.

74

<id>
Value

FindUMsb

Unsigned-integer most-significant bit.

Results in the bit number of the most-significant 1-bit in the binary representation of *Value*. If *Value* is 0, the result has all bits set (e.g., -1 if interpreted as signed).

Result Type and the type of *Value* must both be integer scalar or integer vector types. *Result Type* and operand types must have the same number of components with the same component width. Results are computed per component.

This instruction is currently limited to 32-bit width components.

75

<id>
Value

InterpolateAtCentroid

Result is the value of the input *interpolant* sampled at a location inside both the fragment and the primitive being processed. The value obtained would be the same value assigned to the input variable if it were decorated as **Centroid**.

The operand *interpolant* must be a pointer to the **Input** Storage Class.

The operand *interpolant* must be a pointer to a scalar or vector whose component type is 32-bit floating-point.

This instruction is only valid in the **Fragment** execution model.

Result Type and the type that *interpolant* points to must be the same type.

Use of this instruction requires declaration of the **InterpolationFunction** capability.

76

<id>
interpolant

InterpolateAtSample

Result is the value of the input *interpolant* variable at the location of sample number *sample*. If sample *sample* does not exist, the position used to interpolate the input variable is *poison*.

The operand *interpolant* must be a pointer to the **Input** Storage Class.

The operand *interpolant* must be a pointer to a scalar or vector whose component type is 32-bit floating-point.

This instruction is only valid in the **Fragment** execution model.

The *sample* operand must be a scalar 32-bit integer.

Result Type and the type that *interpolant* points to must be the same type.

Use of this instruction requires declaration of the **InterpolationFunction** capability.

77	<id> <i>interpolant</i>	<id> <i>sample</i>
----	----------------------------	-----------------------

InterpolateAtOffset

Result is the value of the input *interpolant* variable sampled at an offset from the center of the fragment specified by *offset*. The two floating-point components of *offset*, give the offset in pixels in the x and y directions, respectively. An *offset* of (0, 0) identifies the center of the fragment. The range and granularity of offsets supported are implementation-dependent.

The operand *interpolant* must be a pointer to the **Input** Storage Class.

The operand *interpolant* must be a pointer to a scalar or vector whose component type is 32-bit floating-point.

This instruction is only valid in the **Fragment** execution model.

The *offset* operand must be a vector of 2 components of 32-bit floating-point type.

Result Type and the type that *interpolant* points to must be the same type.

Use of this instruction requires declaration of the **InterpolationFunction** capability.

78	<id> <i>interpolant</i>	<id> <i>offset</i>
----	----------------------------	-----------------------

NMin

Result is y if $y < x$, otherwise x. -0 compares less than +0. If one operand is a NaN, the other operand is the result. If both operands are NaN, the result is a NaN.

The operands must all be a scalar or vector whose component type is floating-point.

Result Type and the type of all operands must be the same type. Results are computed per component.

79	<code><id></code> <i>x</i>	<code><id></code> <i>y</i>
----	-------------------------------------	-------------------------------------

NMax

Result is *y* if $x < y$, otherwise *x*. -0 compares less than +0. If one operand is a NaN, the other operand is the result. If both operands are NaN, the result is a NaN.

The operands must all be a scalar or vector whose component type is floating-point.

Result Type and the type of all operands must be the same type. Results are computed per component.

80	<code><id></code> <i>x</i>	<code><id></code> <i>y</i>
----	-------------------------------------	-------------------------------------

NClamp

Result is $\min(\max(x, \text{minVal}), \text{maxVal})$. The resulting value is *poison* if $\text{minVal} > \text{maxVal}$. The semantics used by `min()` and `max()` are those of NMin and NMax.

The operands must all be a scalar or vector whose component type is floating-point.

Result Type and the type of all operands must be the same type. Results are computed per component.

81	<code><id></code> <i>x</i>	<code><id></code> <i>minVal</i>	<code><id></code> <i>maxVal</i>
----	-------------------------------------	--	--

Chapter 3. Appendix A: Changes

3.1. Changes from Version 0.99, Revision 1

- Fork the revision stream, changes section, etc. from the core specification, so this specification has its own, starting numbering at revision 1. This document now lives independently.
- Added integer versions of **abs**, **sign**, **min**, **max**, and **clamp**.
- Removed **floatBitsToInt**, **floatBitsToUint**, **intBitsToFloat**, and **uintBitsToFloat**; these can be handled with **OpBitcast**.
- Removed **fTransform**, not needed.
- Fixed internal bugs
 - 13721: Add **OpTypeStruct**-result versions of **Modf** and **Frexp**: **ModfStruct** and **FrexpStruct**.
- Fixed public bugs
 - 1322: GLSL.std.450 **frexp** wasn't saying the *exp* argument was a pointer to the result

3.2. Changes from Version 0.99, Revision 2

- Moved **AddCarry**, **SubBorrow**, and **MulExtended** type of instructions to the core specification.
- Added integer variant of **Mix**, creating **FMix** and **IMix** (14480).
- Modified spellings to be more regular (14614).

3.3. Changes from Version 0.99, Revision 3

- Add "N" version of **Min**, **Max**, and **Clamp**, creating a version that favors non-NaN operands over NaN operands.
- Bug 15452 Remove **IMix**.
- Bug 15300 Be more consistent that the **InterpolateAt** instructions take a pointer.
- Bug 14548 Document the **Capability** needed for **Double2x32** and **InterpolateAt** instructions.

3.4. Changes from Version 1.00, Revision 1

- Bug 14548 Document the **Capability** needed for **UnpackDouble2x32**.

3.5. Changes from Version 1.00, Revision 2

- Change **precise** to **NoContraction**

3.6. Changes from Version 1.00, Revision 3

- Allow both 16-bit and 32-bit floating-point types in most places where before only 32-bit floating-point types were allowed. This does not effect whether 16-bit floating point types are allowed, which is selected independently. Since 16-bit types were historically disallowed, this is a backward compatible change.
- Fix Khronos internal issue #109: be more clear for **NMin**/**NMax**: If both operands are NaN, the result is a NaN.

3.7. Changes from Version 1.00, Revision 4

- Be clear about **UnpackHalf2x16** denorm rules.

3.8. Changes from Version 1.00, Revision 5

Fixed:

- Khronos SPIR-V Issue #211: As with **FindSMsb** and **FindUMsb**, **FindILsb** needs 32-bit components.

3.9. Changes from Version 1.00, Revision 6

Fixed:

- Khronos SPIR-V Issue #337: The component types of the operands for **Refract** must all be the same.
- Khronos SPIR-V Issue #331: Correct the types in **ModfStruct**.

3.10. Changes from Version 1.00, Revision 7

Support `SPV_KHR_no_integer_wrap_decoration`, in the **SAbs** instruction.

3.11. Changes from Version 1.00, Revision 8

Fixed:

- Khronos SPIR-V Issue #466: **FAbs** of -0.0 is $+0.0$, **FSign** of -0.0 can be either ± 0.0 . **FMin**, **FMax**, **NMin**, and **NMax** are allowed to return either operand when both are zeros.
- Khronos SPIR-V Issue #458: For **Frexp**, be more clear about negative values, and also about which returned value is being discussed.

3.12. Changes from Version 1.00, Revision 9

- Corrected the output range of **Atan**.

3.13. Changes from Version 1.00, Revision 10

- State what **FSign** of $\pm NaN$ is.

3.14. Changes from Version 1.00, Revision 11

- Khronos SPIR-V Issue #555: Deprecate **Modf**, use **ModfStruct** instead. Deprecate **Frexp**, use **FrexpStruct** instead.
- Khronos SPIR-V Issue #284: Say all bits are set, instead of saying -1, for some results of **FindILsb**, **FindSMsb**, and **FindUMsb**.
- Khronos SPIR-V MR #181: Use "fragment" instead of "pixel" in **InterpolateAtCentroid**, **InterpolateAtSample**, and **InterpolateAtOffset**.

3.15. Changes from Version 1.00, Revision 12

- Khronos SPIR-V Issue #705: Clarify the computation provided for **Reflect** is used regardless of input normalization.

3.16. Changes from Version 1.00, Revision 13

- Clarify behavior for **Round**, **RoundEven**, **Trunc**, **FAbs**, **FSign**, **Floor**, **Ceil**, **Fract**, **Sin**, **Cos**, **Tan**, **Asin**, **Acos**, **Sinh**, **Cosh**, **Tanh**, **Asinh**, **Acosh**, **Exp**, **Log**, **Exp2**, **Log2**, **Sqrt**, **InverseSqrt**, **ModfStruct**, **FMin**, **FMax**, **NMin**, **NMax** for special floating-point values (only observable when `SPV_KHR_float_controls2` is used).

3.17. Changes from Version 1.00, Revision 14

- Khronos SPIR-V Issue #800: Clarify behavior for **FMin**, **FMax**, **NMin**, **NMax** to match IEEE 754-2019 (observable when `SPV_KHR_float_controls2` is used).
- Khronos SPIR-V Issue #801: Clarify that the `exp` operand to **Ldexp** is always interpreted as signed.
- Khronos SPIR-V Issue #795: Clarify that **RelaxedPrecision** only applies to the conversion part of the pack and unpack operations.

3.18. Changes from Version 1.00, Revision 15

- Khronos SPIR-V Issue #855: Clarify exponent limits for half-precision floats in **Ldexp**.

3.19. Changes from Version 1.00, Revision 16

- Khronos SPIR-V Issue #912: Clarify that a floating-point type in this specification is an **OpTypeFloat** using the IEEE 754 encoding.